

Adding a new domain

Manufacturing Hybrid GraphRAG — strict onboarding reference. Drop locations, supported file types, ACL-by-folder, schema declaration, the four-extractor stack, end-to-end flow, commands, output artefacts, and a worked example.

1. Inputs

Drop location: `doc_pipeline/input_docs/` — loaded recursively via `Path.rglob("**")` at `doc_pipeline/document_ingestion.py:250`.

Supported file types

Parsers wired at `doc_pipeline/document_ingestion.py:215-218`. Anything else is rejected with *Unsupported file type: <ext>*. There is no `.md` / `.csv` / `.docx` / `.html` parser — convert to `.txt` first (see `scripts/ingest_piston_work_orders.py` for the pattern).

Extension	Parser	What is extracted
.pdf	PDFParser (pdfplumber)	Per-page text + table cells; <code>metadata.page</code> , <code>metadata.has_tables</code>
.txt	TXTParser	Whole file as one document
.xlsx / .xls	ExcelParser (openpyxl)	One document segment per sheet; <code>metadata.sheet_name</code> , <code>metadata.columns</code> , <code>metadata.row</code>

ACL / classification by folder name

Defined at `core/document_acl.py:60`. The classification is attached to chunk metadata at ingestion; retrievers filter against the caller's role.

Subfolder segment	Classification	Visible to roles
management/ or confidential/	confidential	plant_manager, procurement_manager
restricted/ or internal/	restricted	checker tier + above
anything else (e.g. aviation/, work_orders/)	public	everyone

2. Schema declaration

File: `schemas/manufacturing.yaml` (override path via env `KG_SCHEMA_PATH`). Two YAML keys you edit per domain:

entity_types

Each entry needs **name**. Then *one* of:

- **vocabulary:** `[phrase, phrase, ...]` — closed list. The KeywordExtractor matches these phrases verbatim in chunk text and emits Mentions where `identifier == phrase`.
- **id_pattern:** `<regex>` — closed by regex. The CodeExtractor and the adapter's EQUIPMENT_RE lift matches out of text; the schema validator checks them.
- **Neither** → open-vocab. Only the NarrativeExtractor will produce Mentions (low confidence) here.

Optional: **case_sensitive: false** makes both `vocabulary` and `id_pattern` case-insensitive.

edge_types

Each entry needs **name** (uppercase by convention), **source** (entity type name or list), **target** (entity type name or list).

Optional: **min_cardinality**, **max_cardinality**.

traversal_routes

Used by the orchestrator to decide which edges to expand for a given query intent. Each route declares **seed_types** and **walk** (an ordered list of edge_type names).

3. Making KeywordExtractor emit edges (not just Mentions)

The KeywordExtractor emits an EdgeCandidate equipment_id → vocab_phrase only when:

1. The chunk's metadata.equipment_ids is populated, **and**
 2. The schema declares an edge type whose **source** includes Equipment and **target** includes the vocab's entity type.
- metadata.equipment_ids is populated by pipeline/adapter.py:_extract_entity_metadata — it regex-scans chunk text with EQUIPMENT_RE. So the chunk text must contain a string matching that regex, or upstream must hand-set equipment_ids on the metadata.

If your new domain has its own asset ID convention (e.g. MED-IMG-NNNN for medical imagers), you must:

1. Add that pattern to the **Equipment id_pattern** in schemas/manufacturing.yaml.
2. Add the same pattern to **EQUIPMENT_RE** in pipeline/adapter.py:21.
3. Ensure documents contain the ID string in their body text.

This is the only pure-Python edit per domain. Everything else is YAML.

4. Extractor stack (runs on every chunk)

Order matters — earlier extractors win ties (core/knowledge_graph.py:72).

#	Extractor	Author tag	Conf.	Reads	Emits
1	CodeExtractor	system:code	1.00	Chunk text via schema id_pattern regex	Mentions only
2	MetadataExtractor	system:metadata	0.95	metadata.(equipment_ids, alarm_codes, part_numbers, fault_codes)	Mentions + nodes, not edges (Equipment→Alarm, E
3	KeywordExtractor	system:keyword	0.95	Chunk text against every vocabulary	Mentions + schema vocab hit + Equipment→<vocab> edges vi
4	NarrativeExtractor	system:llm_extrac	0.50	Chunk text via regex (_SYMPTOM_PATTERNS, PROCEDURE_PATTERNS)	Mentions, Symptoms, PROCEDURES, PATIENTS; Equipmen

The schema validates every Mention (Schema.validate_entity) and every EdgeCandidate (Schema.validate_edge). Rejects land in kg._rejected — inspectable for diagnosing schema drift.

5. Pipeline flow end-to-end

```
doc_pipeline/input_docs/<your-domain>/*.{pdf,txt,xlsx}
|
v DocumentIngestion.ingest_directory()          (one Document per file/page/sheet)
HybridChunker.chunk_documents()                 (recursive / semantic / sliding_window)
|
v EmbeddingPipeline.build_index()
|   |- bge-small-en-v1.5 (384-dim) encodes every chunk
|   |- Qdrant upsert -> doc_pipeline/vector_store/qdrant/
|   |- Saves doc_pipeline/vector_store/manufacturing_index.manifest.json
|   |- Saves doc_pipeline/vector_store/manufacturing_index_chunks.json
|
v pipeline/adapter.py:chunks_to_core_docs()
|   |- stable_chunk_id(source, idx)
|   |- regex-extracts equipment_ids / alarm_codes / part_numbers / fault_codes
|   |   from chunk text into metadata
|   |- outputs [{chunk_id, text, metadata}, ...]
|
v KnowledgeGraph.build_from_documents()
|   |- runs all 4 extractors on each doc
|   |- validates Mentions + edges against schema
|   |- stamps Provenance{author, confidence, source_chunk_id, timestamp}
|   |- Saves data/processed/knowledge_graph.json
```

6. Commands

Goal	Command
Add a new domain's docs	Drop files under doc_pipeline/input_docs/<domain>/ (use a sub-folder for ACL)
Convert JSONL/CSV/other to .txt	Write a script under scripts/ modeled on scripts/ingest_piston_work_orders.py (deterministic filenames, b
Edit the ontology	Edit schemas/manufacturing.yaml — add entity types with vocabulary / id_pattern, add edge types, add
Extend Equipment regex (rare)	Edit pipeline/adapter.py:EQUIPMENT_RE
Full rebuild (index + KG)	python main.py --rebuild --no-llm
Rebuild + run a query	python main.py --rebuild --query "your question"
Rebuild + diagnostic (LLM)	python main.py --rebuild --diagnostic "your question"
Inspect resulting KG	python -c "from core.knowledge_graph import KnowledgeGraph; kg=KnowledgeGraph(); kg.load(); print(
Inspect schema rejects	After build: kg._rejected (list of dicts with kind, entity_type/relation, chunk_id, author)
Launch Streamlit + FastAPI	./run.sh (FastAPI on :8000, Streamlit on :8501)
Stop	./stop.sh

7. Output artefacts (what gets written)

Path	Content	Persistence
doc_pipeline/vector_store/qdrant/	Qdrant collection "manufacturing_corpus" (embedding_model, dim) — sanitized across runs; --rebuild overwrites	Survives
doc_pipeline/vector_store/manufacturing_index_collections.json	{collection_name, embedding_model, dim} — sanitized	Overwritten on rebuild
doc_pipeline/vector_store/manufacturing_index_collections.json	Links to chunks with text + metadata (BM25 / KG source)	Overwritten on rebuild
data/processed/knowledge_graph.json	NetworkX graph snapshot + provenance	Overwritten on rebuild
data/processed/audit.sqlite	HITL approval log + LangGraph checkpoints	Append-only

8. Minimal worked example (medical-device domain)

schemas/manufacturing.yaml — additions

```
entity_types:
- name: Equipment
  id_pattern: '<existing>|^MED-IMG-\d{4}$'          # add med-imager IDs
  case_sensitive: false

- name: Component                                # extend existing vocab
  case_sensitive: false
  vocabulary:
    - <existing terms...>
    - x-ray tube
    - detector panel
    - gantry motor
    - high-voltage generator

- name: Cause
  case_sensitive: false
  vocabulary:
    - <existing terms...>
    - calibration drift
    - detector burn-in
    - cooling failure
```

pipeline/adapter.py — single regex edit

```
EQUIPMENT_RE = re.compile(r'...existing...|^MED-IMG-\d{4}$')
```

Stage data + rebuild + verify

```
# stage data
mkdir -p doc_pipeline/input_docs/medical/{manuals,work_orders}
cp /path/to/service_manuals/*.pdf doc_pipeline/input_docs/medical/manuals/
# convert your JSONL/CSV of work orders to .txt with the same shape as
# scripts/ingest_piston_work_orders.py

# rebuild
python main.py --rebuild --no-llm

# verify
python -c "
from core.knowledge_graph import KnowledgeGraph
kg = KnowledgeGraph(); kg.load()
from collections import Counter
c = Counter(d.get('entity_type') for _,d in kg.graph.nodes(data=True))
print(c)
print('rejects:', len(kg._rejected))
"
```

9. What you CANNOT do without code changes

Wish	Why it requires Python
Ingest .md / .docx / .csv / .html	Only .pdf/.txt/.xlsx/.xls have parsers — add a class to doc_pipeline/document_ingestion.py and recompile
Custom entity-ID convention auto-tagged from pipeline/adapter.py	EQUIPMENT_RE is hard-coded
Aliasing (e.g. "mag" and "magneto" -> same Keyphrase)	Keyphrase Extractor uses the vocab phrase as the canonical identifier. For aliases, either add each as a separate phrase or use a custom extractor
LLM-driven open-vocab Symptom/Procedure Narrative	Narrative Extractor is regex-only; the interface accepts any Extractor subclass — swap it out (default is Keyphrase)
New extractor confidence tier (e.g. CMMS import)	Subclass core/kg/extractors/base.Extractor, set author=ProvenanceAuthor.IMPORT_CMMS, register

Bottom line. Schema YAML + a converter script + `python main.py --rebuild` covers every domain whose data fits the supported file types and uses an Equipment ID convention you have declared.